

Day 1 — code map

One row per file in `backend/goldset/`. What each does, the one function that carries it, and the decision inside it I would have to defend in a review.

This is a map, not a manual. No signatures, no snippets, no argument lists — read the module docstrings for those; they are written for that purpose.

file	what it does	the function that matters	the decision I would have to defend
<code>rules.py</code>	Holds the four drop rules and turns a per-rule scorecard into a single verdict.	<code>verdict_for</code>	The rules live in one module that the screener, the auditor and the manual verifier all import rather than each restating them. The audit's agreement number is only meaningful if the human and the model were asked the same four questions, so retyping the rules anywhere would silently invalidate it. Drop beats fix, because a question that is both vague and verbatim is not worth rephrasing.
<code>console.py</code>	Terminal and JSONL helpers shared by the three interactive CLIs.	<code>append_jsonl</code>	Every decision is appended to disk one row at a time, before the next screen renders. It costs a file handle per keypress and makes all three CLIs resumable, so an interrupted 137-row pass costs a minute rather than the pass. It is also why <code>audit_decisions.jsonl</code> is the durable record and the queue files are disposable.
<code>generate.py</code>	Drafts one question per sampled chunk with an LLM, and checks which chunk actually answers it. Writes drafts, never labels.	<code>verify_label</code>	The gold label follows a second, <code>temperature=0</code> call asking which of the chunk and its two document neighbours contains the answer — not the chunk the question was drafted from. 12 of the 115 gold rows were relabelled by this check. Where it cannot pick, the row is stamped <code>unverified</code> and never offered for screening (9 rows), rather than guessed at.
<code>generate.py</code> – <code>the pool</code>	<i>(same file)</i> Decides which chunks can carry a question at all.	<code>select_eligible</code>	Four structural filters cut 1,480 chunks to 392, and the scaffolding filter was added after the first audit, because 7 of 14 human rejections were questions about a page's <code>title:</code> . That is a filter changed in response to measured data, which is right — and it is also why the eligible pool moved from 405 to 392 and the drafting sample is no longer re-drawable. Both halves have to be said together.
<code>screen.py</code>	Scores every draft against the four rules with one LLM call per rule, and rewrites the fixable ones.	<code>screen_draft</code>	Rule 2 never goes to the model. "More than four consecutive words" is an exact property of two strings and a longest-common-run scan answers it identically on every run; an LLM counting words is guessing, and its guess would be the one number in this pipeline that could not be reproduced. Every rule is scored even after a drop is certain, because short-circuiting would hollow out the per-rule counts.
<code>audit.py</code>	The human adjudication CLI: presents a question and its chunk, records the verdict, and computes the agreement arithmetic.	<code>render_question</code>	This function is the entire credibility claim. It reads the question, the source doc, the chunk id and the fixed rule text, and nothing the screener decided — not even indirectly. The rule text is shown in full and in fixed order for every row, so its presence carries no signal about which rule fired. Four tests pin it, including one that fills every <code>screen_*</code> field with marker strings and asserts none reaches the terminal.

file	what it does	the function that matters	the decision I would have to defend
<code>audit.py</code> — the queue	(<i>same file</i>) Chooses which rows a human is asked to decide.	<code>build_coverage_queue</code>	Membership is decided by calling <code>assemble.adjudicate</code> , not by a second copy of its rules. That function is what actually builds the gold set, so asking it directly is the only way the queue cannot drift out of step with the file it covers. <code>--cover-rejections</code> extends it to rows dropped on screener agreement alone: a wrongly kept row can be found by reading the gold set, a wrongly dropped one cannot.
<code>assemble.py</code>	Settles one verdict per screened row, applies the multi-label map, and writes the gold set plus the accept/reject census.	<code>adjudicate</code>	The precedence order, highest first: hand exclusions, then my audit verdict, then both screeners agreeing, then exclusion. Hand exclusions outrank the audit — judging a question good does not make the row reproducible, and those three rows were removed because a clean re-run could not produce them. Rows the screeners split on that nobody read are excluded rather than guessed, and counted in the report either way.
<code>multilabel.py</code>	Flags every other chunk sharing an 8+ word sentence with a gold chunk, as a candidate for an equally-correct label.	<code>candidates_for</code>	Nothing in this file may write <code>data/multilabel.json</code> . The rule is deliberately over-broad — 242 candidates across 92 of 115 rows, of which 26 rows survived hand review — because a shared sentence is evidence of duplication, not of an answer. Auto-accepting would put chunks in <code>gold_chunk_ids</code> that do not answer the question, inflating recall for every lane at once, invisibly. The <code>verdict</code> field is emitted null and never pre-filled.
<code>split.py</code>	Splits the gold set 70/30 into train and test at seed 42, and refuses to produce a leaking split.	<code>group_by_shared_chunk</code>	Rows that share a gold chunk travel together, and the relation is transitive, so this is a component walk rather than a pairwise check. Multi-labelling makes a row-wise shuffle unsafe: it put 8 chunks on both sides. Groups are then placed largest-first into whichever side is furthest below quota, because filling in shuffled order let one late 5-row group push the realised split to 63/37.
<code>verify.py</code>	The fully-manual alternative to screen + assemble: read every draft, decide every row.	<code>run</code>	It is kept although it is no longer the default path, because it is the ground truth the screener is measured against. It writes <code>status: "verified"</code> ; <code>assemble.py</code> writes <code>status: "screened"</code> . Nothing downstream may confuse a row a human built from scratch with a row a human adjudicated a screener's verdict on, and that distinction lives in the file rather than only in the README.
<code>__init__.py</code>	Empty. Package marker only.	—	Nothing to defend, and that is the decision: importing <code>backend.goldset</code> must not pull in a Groq client, a tokenizer or a database connection as a side effect.

Two files that are not in this table but decide what these produce

`backend/config.py` holds every configuration literal — chunk size 512, overlap 64, model ids — because none may appear in application code. `backend/retrieval/dense_store.py` is the only thing that talks to the chunk store; `generate.py`, `screen.py`, `audit.py`, `verify.py` and `multilabel.py` all read chunks through it and none opens a connection of its own.

Where the tests are

`backend/tests/` carries one module per gold-set file: `test_goldset_generate.py`, `test_goldset_screen.py`, `test_goldset_assemble.py`, `test_goldset_audit.py`, `test_goldset_split.py`, `test_goldset_verify.py`. Groq and the store are stubbed throughout — `pytest` passes with no network and no API keys.
